

(<https://metana.io/>)

Book a call >

Claim Your **Black Friday Deal!**

Add a Career to Your Cart at 20% Off

02
Days

08
Hours

23
Minutes

20
Seconds

Claim Offer

Top 9 Smart Contract Programming Languages In 2025

👤 Metana Editorial(<https://metana.io/blog/author/editorial/>) | 📅 October 10, 2025

💎 Smart Contracts (<https://metana.io/blog/category/smart-contracts/>)

TLDR; Smart Contract Languages

1. **Solidity** – Ethereum/EVM chains, most popular, beginner-friendly
2. **Vyper** – Ethereum/EVM, Python-like, security-focused
3. **Rust** – Solana/Polkadot/Near, high-performance, advanced
4. **Cairo** – StarkNet L2, zero-knowledge proofs, cutting-edge
5. **Move** – Aptos/Sui, resource-oriented, asset safety
6. **Clarity** – Stacks/Bitcoin L2, decidable, Bitcoin integration
7. **Motoko** – Internet Computer, actor model, persistent storage
8. **Cadence** – Flow blockchain, NFT-focused, resource-oriented
9. **Ink!** – Polkadot parachains, Rust-based DSL, cross-chain ready

What is a Solidity Smart Contract?

A **Solidity smart contract** (<https://metana.io/blog/solidity-functions-types-and-use-cases/>) is a program that runs on the **Ethereum blockchain** (<https://metana.io/blog/what-is-ethereum-2-0-eth-2-0-and-how-has-it-changed-the-world/>), allowing transactions and agreements to execute automatically when predefined conditions are met. Written in the **Solidity** programming language, these contracts are self-executing with the terms of the agreement directly embedded in the code. Once deployed to the blockchain, smart contracts are immutable and transparent, meaning they can't be changed or tampered with. They enable secure and efficient interactions between parties without needing intermediaries, making them a key component in decentralized applications (dApps), **decentralized finance (DeFi)** (<https://metana.io/blog/what-is-defi-and-how-does-it-work>), token creation, and more.

9 Smart Contract Programming Languages in 2025

1. Solidity (<https://metana.io/blog/learn-solidity-understanding-the-basics/>)

Platform: Ethereum Virtual Machine (EVM) and EVM-compatible chains

Market Share: ~65% of all smart contracts

Difficulty: Beginner to Intermediate

Key Features

- **Static Typing:** Compile-time error detection and type safety
- **Contract Inheritance:** Object-oriented programming with inheritance support
- **Modifier System:** Reusable code blocks for access control and validation
- **Assembly Integration:** Inline assembly for gas optimization



- **Event Logging:** Built-in event system for dApp integration

Code Example

```
pragma solidity ^0.8.19;

contract TokenVault {
    mapping(address => uint256) private balances;

    modifier onlyPositiveAmount(uint256 amount) {
        require(amount > 0, "Amount must be positive");
        _;
    }

    function deposit() external payable onlyPositiveAmount(msg.value) {
        balances[msg.sender] += msg.value;
        emit Deposited(msg.sender, msg.value);
    }

    event Deposited(address indexed user, uint256 amount);
}
```

Strengths

- Largest developer community and extensive documentation
- Rich ecosystem with frameworks like Hardhat, Truffle, and Foundry
- Wide blockchain compatibility (Ethereum, Polygon, BSC, Avalanche)
- Mature debugging tools and security analyzers

Weaknesses

- Susceptible to reentrancy and integer overflow attacks
- Higher gas costs compared to some alternatives
- Complex syntax can lead to developer errors

Best For

- DeFi protocols, NFT marketplaces, DAOs, and multi-chain applications

2. Vyper

Platform: Ethereum Virtual Machine (EVM)

Market Share: ~8% of EVM smart contracts

Difficulty: Beginner to Intermediate

Key Features

- **Python-like Syntax:** Clean, readable code structure
- **Security by Design:** Eliminates dangerous features like recursion and infinite loops
- **Bounded Loops:** All loops must have a maximum iteration count
- **Overflow Protection:** Built-in SafeMath equivalent functionality
- **Simplified Inheritance:** Single inheritance only to reduce complexity

Code Example



```
# @version ^0.3.7

event Transfer:
    sender: indexed(address)
    receiver: indexed(address)
    amount: uint256

balances: HashMap[address, uint256]
total_supply: public(uint256)

@external
def transfer(receiver: address, amount: uint256) → bool:
    assert self.balances[msg.sender] ≥ amount, "Insufficient balance"
    self.balances[msg.sender] -= amount
    self.balances[receiver] += amount
    log Transfer(msg.sender, receiver, amount)
    return True
```

Strengths

- More secure by default due to restricted feature set
- Lower gas consumption than Solidity in many cases
- Easier to audit due to simplified syntax
- Strong compile-time checks prevent common vulnerabilities

Weaknesses

- Limited ecosystem compared to Solidity
- Fewer advanced features for complex applications
- Smaller developer community and fewer learning resources

Best For

- Security-critical financial contracts, [simple DeFi protocols \(https://metana.io/blog/understanding-defi-smart-contracts-beginners-guide/\)](https://metana.io/blog/understanding-defi-smart-contracts-beginners-guide/), and educational projects

3. Rust

Platform: [Solana \(https://metana.io/blog/why-learn-solana-is-it-worth-it/\)](https://metana.io/blog/why-learn-solana-is-it-worth-it/), Polkadot, Near Protocol, Cosmos

Market Share: ~15% of [non-EVM smart contracts \(https://metana.io/blog/interacting-with-the-evm/\)](https://metana.io/blog/interacting-with-the-evm/).

Difficulty: Intermediate to Advanced

Key Features

- **Memory Safety:** Zero-cost abstractions with compile-time memory management
- **Performance:** Near C++ speed with safety guarantees
- **Concurrency:** Built-in support for parallel processing
- **Pattern Matching:** Powerful control flow with match expressions
- **Cargo Ecosystem:** Rich package manager and build system

Code Example (Solana)



```

use solana_program::{
    account_info::{next_account_info, AccountInfo},
    entrypoint,
    entrypoint::ProgramResult,
    program_error::ProgramError,
    pubkey::Pubkey,
};

entrypoint!(process_instruction);

pub fn process_instruction(
    program_id: &Pubkey,
    accounts: &[AccountInfo],
    instruction_data: &[u8],
) → ProgramResult {
    let accounts_iter = &mut accounts.iter();
    let account = next_account_info(accounts_iter)?;

    if !account.is_signer {
        return Err(ProgramError::MissingRequiredSignature);
    }

    // Process the instruction
    match instruction_data[0] {
        0 => initialize_account(account),
        1 => transfer_tokens(accounts, instruction_data),
        _ => Err(ProgramError::InvalidInstructionData),
    }
}

```

Strengths

- Exceptional performance and low-level control
- Strong type system prevents many runtime errors
- Growing ecosystem with excellent tooling
- Cross-platform compatibility

Weaknesses

- Steep learning curve for developers new to systems programming
- Longer development time due to strict compiler
- Fewer blockchain-specific libraries compared to Solidity

Best For

- High-performance DeFi protocols (<https://metana.io/blog/understanding-defi-smart-contracts-beginners-guide/>), gaming applications, and infrastructure projects

4. Cairo

Platform: StarkNet (Ethereum Layer 2)

Market Share: ~3% but rapidly growing

Difficulty: Advanced

Key Features

- **STARK Proofs:** Native support for scalable, transparent cryptographic proofs
- **Cairo VM:** Custom virtual machine optimized for proving
- **Field Arithmetic:** Built-in support for finite field operations
- **Provable Programs:** Mathematical guarantees of execution correctness
- **Zero-Knowledge Privacy:** Computations can be verified without revealing inputs



Code Example

```
%lang starknet
from starkware.cairo.common.cairo_builtins import HashBuiltin
from starkware.starknet.common.syscalls import get_caller_address

@storage_var
func balance(user: felt) → (res: felt) {
}

@external
func deposit{syscall_ptr: felt*, pedersen_ptr: HashBuiltin*, range_check_ptr}(
    amount: felt
) {
    let (caller) = get_caller_address();
    let (current_balance) = balance.read(caller);
    balance.write(caller, current_balance + amount);
    return ();
}
```

Strengths

- Cutting-edge zero-knowledge technology
- Massive scalability improvements (1000x+ throughput)
- Mathematical proof of correctness
- Lower transaction costs on Layer 2

Weaknesses

- Extremely steep learning curve
- Limited tooling and development resources
- Complex debugging and testing
- Requires deep cryptography knowledge

Best For

- Privacy-preserving applications, high-frequency trading, and scalable DeFi

5. Move

Platform: Aptos, Sui, originally Diem

Market Share: ~5% of total smart contract deployments

Difficulty: Intermediate to Advanced

Key Features

- **Resource-Oriented Programming:** Assets are first-class resources with move semantics
- **Linear Types:** Resources cannot be copied or accidentally lost
- **Formal Verification:** Built-in support for mathematical proofs
- **Module System:** Clean organization and code reuse
- **Bytecode Verification:** Runtime safety checks

Code Example



```

module 0x1::SimpleWallet {
  use std::signer;
  use aptos_framework::coin::{Self, Coin};
  use aptos_framework::aptos_coin::AptosCoin;

  struct Wallet has key {
    balance: Coin<AptosCoin>,
  }

  public entry fun create_wallet(account: &signer) {
    let wallet = Wallet {
      balance: coin::zero<AptosCoin>(),
    };
    move_to(account, wallet);
  }

  public entry fun deposit(account: &signer, amount: u64) acquires Wallet {
    let wallet = borrow_global_mut<Wallet>(signer::address_of(account));
    let deposit_coin = coin::withdraw<AptosCoin>(account, amount);
    coin::merge(&mut wallet.balance, deposit_coin);
  }
}

```

Strengths

- Revolutionary resource safety model
- Prevents common asset management bugs
- Strong formal verification capabilities
- Clean, modern syntax design

Weaknesses

- Limited blockchain ecosystem adoption
- Fewer developers familiar with resource-oriented programming
- Still evolving tooling and documentation

Best For

- Asset management systems, NFT platforms, and financial applications requiring strict resource safety

6. Clarity

Platform: Stacks (Bitcoin Layer 2)

Market Share: ~2% but growing with Bitcoin DeFi

Difficulty: Intermediate

Key Features

- **Decidable Language:** Static analysis can determine all possible execution paths
- **Bitcoin Integration:** Direct access to Bitcoin state and transactions
- **Type Safety:** Strong typing system prevents runtime errors
- **Interpreted Execution:** No compilation step, direct source code execution
- **Predictable Costs:** Gas costs are deterministic and calculable

Code Example



```
(define-data-var counter uint 0)

(define-public (increment)
  (let ((current-value (var-get counter)))
    (var-set counter (+ current-value 1))
    (ok (var-get counter))))

(define-read-only (get-counter)
  (var-get counter))

(define-public (transfer-stx (amount uint) (recipient principal))
  (stx-transfer? amount tx-sender recipient))
```

Strengths

- Unique Bitcoin integration capabilities
- Complete decidability enables perfect static analysis
- No compiler bugs since code is interpreted
- Growing Bitcoin DeFi ecosystem

Weaknesses

- Limited to Stacks ecosystem
- Lisp-like syntax may be unfamiliar to many developers
- Smaller community and fewer resources

Best For

- Bitcoin-based DeFi, cross-chain Bitcoin applications, and projects requiring formal verification

7. Motoko

Platform: Internet Computer (IC)

Market Share: ~1% but unique capabilities

Difficulty: Intermediate

Key Features

- **Actor Model:** Native support for concurrent, distributed computing
- **Orthogonal Persistence:** Automatic state management without databases
- **Candid Integration:** Type-safe inter-service communication
- **WebAssembly Target:** High-performance execution environment
- **Async/Await:** Modern asynchronous programming patterns

Code Example



```
import Time "mo:base/Time";
import Map "mo:base/HashMap";

actor TokenLedger {
  private stable var totalSupply : Nat = 0;
  private var balances = Map.HashMap<Principal, Nat>(10, Principal.equal, Principal.hash);

  public func transfer(to: Principal, amount: Nat) : async Bool {
    let caller = msg.caller;
    switch (balances.get(caller)) {
      case null { false };
      case (?balance) {
        if (balance ≥ amount) {
          balances.put(caller, balance - amount);
          switch (balances.get(to)) {
            case null { balances.put(to, amount); };
            case (?toBalance) { balances.put(to, toBalance + amount); };
          };
          true
        } else { false }
      };
    }
  };
}
```

Strengths

- Revolutionary orthogonal persistence
- Built for internet-scale applications
- Native multi-canister architecture
- Strong type system with modern features

Weaknesses

- Tied exclusively to Internet Computer
- Smaller ecosystem and developer base
- Different programming paradigms may require learning curve

Best For

- Large-scale dApps (<https://metana.io/blog/top-languages-for-dapp-development-beginner-guide/>), social media platforms, and applications requiring persistent storage

8. Cadence

Platform: Flow Blockchain

Market Share: ~2% focused on NFTs and gaming

Difficulty: Intermediate

Key Features

- **Resource-Oriented Programming:** Similar to Move but with different design philosophy
- **Capability-Based Security:** Fine-grained access control
- **Ergonomic Design:** Developer-friendly syntax and semantics
- **Built-in NFT Support:** Native non-fungible token primitives
- **Type Safety:** Strong static typing with safety guarantees

Code Example



```

pub contract SimpleNFT {
  pub var totalSupply: UInt64

  pub resource NFT {
    pub let id: UInt64
    pub let metadata: {String: String}

    init(metadata: {String: String}) {
      self.id = SimpleNFT.totalSupply
      self.metadata = metadata
      SimpleNFT.totalSupply = SimpleNFT.totalSupply + 1
    }
  }

  pub fun createNFT(metadata: {String: String}): @NFT {
    return ←create NFT(metadata: metadata)
  }

  init() {
    self.totalSupply = 0
  }
}

```

Strengths

- Excellent for NFT and gaming applications
- Resource safety prevents common asset bugs
- Growing ecosystem in digital collectibles
- User-friendly account model

Weaknesses

- Limited to Flow blockchain
- Smaller developer community
- Less DeFi ecosystem compared to Ethereum

Best For

- NFT marketplaces, blockchain games, and consumer-facing applications

9. Ink!

Platform: Polkadot parachains (Substrate-based)

Market Share: ~1% but growing with Polkadot ecosystem

Difficulty: Intermediate to Advanced

Key Features

- **Rust-Based DSL:** Domain-specific language built on top of Rust
- **WebAssembly Compilation:** Compiles to efficient WASM bytecode
- **Substrate Integration:** Native compatibility with Polkadot parachains
- **No Gas Fees:** Uses weight-based fee model instead of gas
- **Cross-Chain Ready:** Built for Polkadot's interoperability vision

Code Example

rust



```

#[cfg_attr(not(feature = "std"), no_std)]

use ink_lang as ink;

#[ink::contract]
mod simple_storage {
    use ink_storage::Mapping;

    #[ink(storage)]
    pub struct SimpleStorage {
        value: i32,
        balances: Mapping<AccountId, Balance>,
    }

    impl SimpleStorage {
        #[ink(constructor)]
        pub fn new(init_value: i32) → Self {
            Self {
                value: init_value,
                balances: Mapping::default(),
            }
        }

        #[ink(message)]
        pub fn get(&self) → i32 {
            self.value
        }

        #[ink(message)]
        pub fn set(&mut self, value: i32) {
            self.value = value;
        }

        #[ink(message, payable)]
        pub fn deposit(&mut self) {
            let caller = self.env().caller();
            let endowment = self.env().transferred_value();
            let current_balance = self.balances.get(&caller).unwrap_or(0);
            self.balances.insert(&caller, current_balance + endowment);
        }

        #[ink(message)]
        pub fn balance_of(&self, owner: AccountId) → Balance {
            self.balances.get(&owner).unwrap_or(0)
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[ink::test]
    fn constructor_works() {
        let storage = SimpleStorage::new(42);
        assert_eq!(storage.get(), 42);
    }
}

```

Strengths

- Leverages Rust's safety and performance benefits
- Native Polkadot ecosystem integration
- Excellent testing framework and tooling
- Growing developer adoption in Substrate ecosystem
- Weight-based fee model more predictable than gas

Weaknesses



- Limited to Polkadot/Substrate ecosystem
- Requires understanding of both Rust and Polkadot concepts
- Smaller community compared to Ethereum languages
- Documentation still maturing

Best For

- Cross-chain DeFi protocols, parachain-specific applications, and projects leveraging Polkadot interoperability

Smart Contract Language Comparison Matrix

Language	Security	Performance	Learning Curve	Ecosystem	Gas Efficiency	Multi-Chain
Solidity	★★★★	★★★★	★★★★★	★★★★★★	★★★★	★★★★★★
Vyper	★★★★★★	★★★★★	★★★★★	★★	★★★★★	★★★★★
Rust	★★★★★	★★★★★★	★★	★★★	★★★★★★	★★★
Cairo	★★★★★★	★★★★★★	★	★★	★★★★★★	★★
Move	★★★★★★	★★★★★	★★	★★	★★★★★	★★
Clarity	★★★★★★	★★★	★★★	★★	★★★	★
Motoko	★★★★★	★★★★★	★★★★	★★	★★★★★	★
Cadence	★★★★★	★★★★★	★★★★	★★	★★★★★	★
Ink!	★★★★★	★★★★★★	★★	★★	★★★★★	★★

Bottom Line

The choice of **smart contract programming language** significantly impacts your project’s success. While Solidity remains the dominant choice for most applications, newer languages offer compelling advantages for specific use cases.

Consider your team’s expertise, project requirements, target blockchain, and long-term maintenance needs when making your decision. The smart contract development landscape rewards both staying current with emerging technologies and mastering established platforms.

Start with one language, build expertise, then expand your toolkit based on project needs and market opportunities.

Frequently Asked Questions

 **Metana Editorial**(<https://metana.io/blog/author/editorial/>)

Powered by Metana Editorial Team, our content explores technology, education and innovation. As a team, we strive to provide everything from step-by-step guides to thought provoking insights, so that our readers can gain impeccable knowledge on emerging trends and new skills to confidently build their career. While our articles cover a variety of topics, we are highly focused on Web3, Blockchain, Solidity, Full stack, AI and Cybersecurity. These articles are written, reviewed and thoroughly vetted by our team of subject matter experts, instructors and career coaches.



Top 9 Smart Contract Programming Languages 2025



LEARN JOB-READY WEB3 SKILLS ON YOUR
SCHEDULE WITH 1-ON-1 SUPPORT & GET A
JOB, OR YOUR MONEY BACK

Which language is most suitable for a smart contract?

✖ (https://metana.io/blog/should-i-learn-python-before-solidity/) N. (https://metana.io/blog/angular-in-2025-still-relevant/) **Solidity** is the most suitable for Ethereum-based smart contracts. For other blockchains, **Rust** (for Solana) and **Vyper** (another Ethereum language) are also popular choices.

Is Solidity harder than Python?

Metana Guarantees a Job

Plus Risk Free 2-Week Refund Policy  **Solidity** is generally considered more difficult than **Python** due to its blockchain-specific nature, need for gas optimization, and the decentralized environment in which it runs.

You're guaranteed a new job in web3—or you'll get a full tuition refund. We also offer a hassle-free two-week refund policy. If you're not satisfied with your purchase for any reason, you can request a refund, no questions asked.

The most advanced Solidity curriculum on the internet!

-  4 Months
-  Prior coding experience required
-  20h/ Week
-  1-on-1 mentorship
-  Expert code reviews
-  Coaching & career services

View Program [>/web3-solidity-bootcamp-ethereum-blockchain/](/web3-solidity-bootcamp-ethereum-blockchain/)

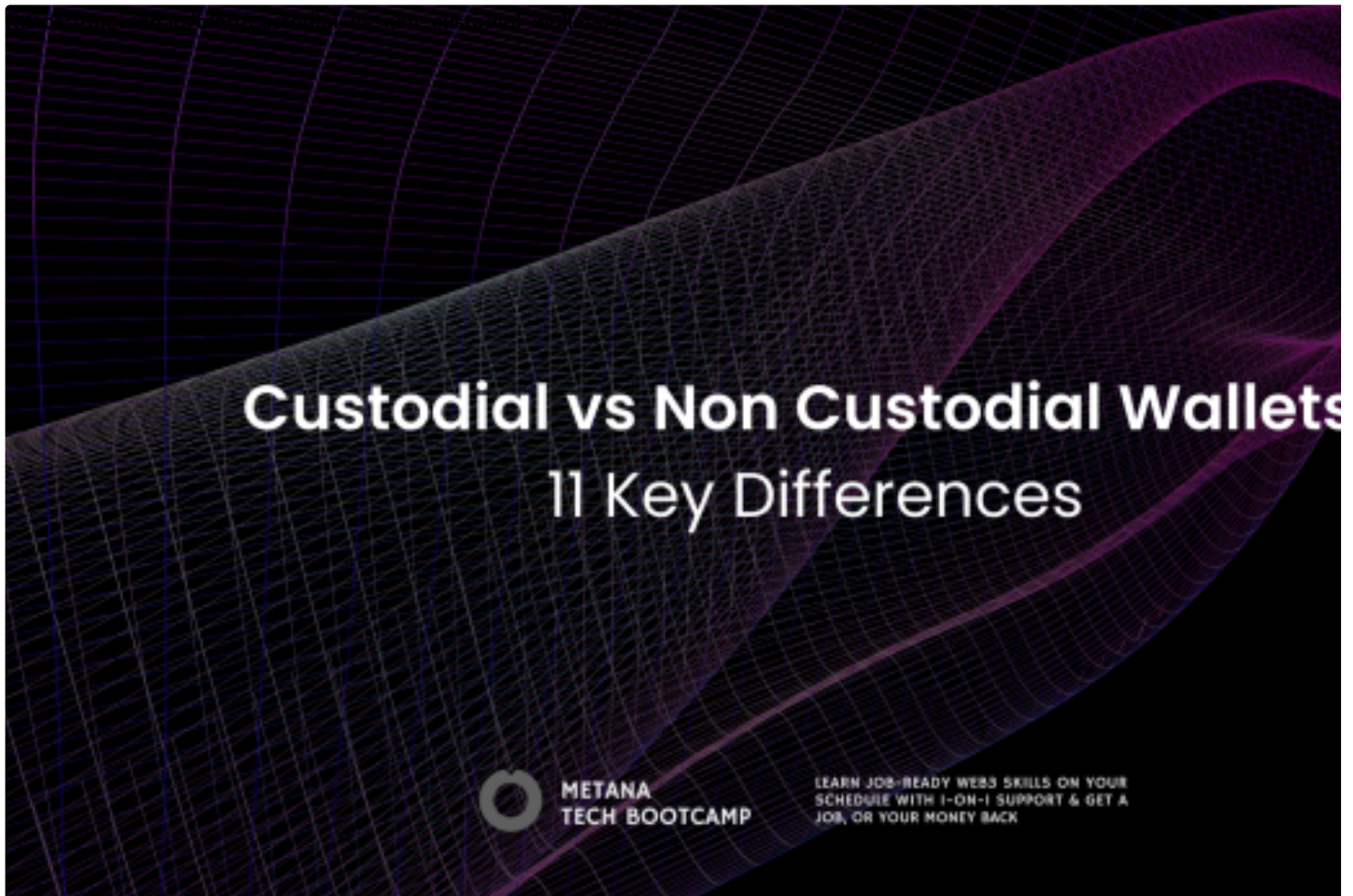
Learn foundational principles while gaining hands-on experience with Ethereum, DeFi, and Solidity.



- 📅 7 Months
- 📊 Beginner - Zero to Hero
- 🕒 25h/ Week
- ✅ Your very own personal support tutor
- ✅ 1-on-1 mentorship
- ✅ Expert code reviews
- ✅ Coaching & career services

[View Program](#) >(/web3-beginner-bootcamp/)

You May Also Like



Metana Editorial (<https://metana.io/blog/author/editorial/>) 5 days ago

Custodial vs Non Custodial Wallets: 11 Key Differences (<https://metana.io/blog/custodial-vs-non-cus>)

Choosing between custodial vs non custodial wallets determines your cryptocurrency security and control. Custodial wallets...



13 Most In-Demand Tech Jobs in 2026



METANA
TECH BOOTCAMP

LEARN JOB-READY WEB3 SKILLS ON YOUR
SCHEDULE WITH 1-ON-1 SUPPORT & GET A
JOB, OR YOUR MONEY BACK



Start your new career today. Talk to an advisor.

Metana Editorial (<https://metana.io/blog/author/metadataeditorial/>) 1 week ago

13 Most In-Demand Tech Jobs in 2026 (<https://metana.io/blog/most-in-demand-tech-jobs-in-2026/>)

Read this article to discover the 13 most in-demand tech jobs for 2026; from AI...

- ✓ Guarantee: Job or 100% money back
- ✓ Expert-curated curriculum
- ✓ On-demand mentor support

First Name *

Last Name *

Email *

Phone *

☐ I consent to marketing emails, calls, and text messages, including those made with an autodialed or artificial voice messages. Message and data rates may apply. Unsubscribe anytime per our Privacy Policy. Consent is not a condition of purchase.

Continue



ADDRESS

44 Montgomery St,
San Francisco, CA 94104, United States

Subscribe to our newsletter

Subscribe

CONTACT US

Call us directly

(415) 416-0800

For questions about bootcamps, tuition and admissions:

admissions@metana.io

For general inquiries, including press and partnership opportunities.

info@metana.io

COMPANY

[Home\(https://metana.io/\)](https://metana.io/)

[Blog\(/blog/\)](/blog/)

[Tuition\(/tuition/\)](/tuition/)

[Metana's Job Guarantee\(/job-guarantee/\)](/job-guarantee/)

[Jobs at Metana\(/jobs/\)](/jobs/)

[Refer a friend\(/refer/\)](/refer/)

[Student Perks\(/perks/\)](/perks/)

[Testimonials\(/reviews/\)](/reviews/)

[2024 Outcomes Report\(https://metana.io/2024-outcome-report/\)](https://metana.io/2024-outcome-report/)

[LMS login\(https://app.metana.io/\)](https://app.metana.io/)

Still Unsure?(<https://app.buildform.ai/forms/PNVFLdNTysjK>)

BOOTCAMP

[Coding Bootcamps\(/coding-bootcamps/\)](/coding-bootcamps/)

- [Software Engineering Bootcamp\(/full-stack-software-engineer-bootcamp/\)](/full-stack-software-engineer-bootcamp/)

- [Software Engineering Career Accelerator\(/advanced-software-engineering-bootcamp/\)](/advanced-software-engineering-bootcamp/)

[Web3 Bootcamps\(/web3-0-bootcamp/\)](/web3-0-bootcamp/)

- [Solidity Bootcamp\(/web3-solidity-bootcamp-ethereum-blockchain/\)](/web3-solidity-bootcamp-ethereum-blockchain/)

- [Web3 Beginner Bootcamp\(/web3-beginner-bootcamp/\)](/web3-beginner-bootcamp/)

- [Rust Bootcamp\(/web3-rust-bootcamp-solana-blockchain/\)](/web3-rust-bootcamp-solana-blockchain/)



Data + AI bootcamps

- Data Analytics Bootcamp(/data-analytics-bootcamp/)
- Data Science Bootcamp(/data-science-bootcamp/)
- AI/ML Bootcamp(/ai-machine-learning-bootcamp/)
- UX/UI Design Bootcamp(/ux-ui-design-bootcamp/)
- Cybersecurity Bootcamp(/cybersecurity-bootcamp/)
- Jobcamp™ - Career Prep(/jobcamp/)
- Career Consultation(/career-consultation/)

FOR BUSINESSES

Business(/business/)

- Hiring Partner(/business/hiring-partner/)
- Upskill Your Team(/business/upskill-your-team/)

LEGAL

Terms of Use(/legal/terms-of-use/)

Privacy Policy(/legal/privacy-policy/)

Job Guarantee Policy(/legal/withdrawal-and-refund-policy/)

Withdrawal and Refund Policy(/legal/withdrawal-and-refund-policy/)

Attributions & Credits(/legal/attributions-and-credits/)

 **CARBON NEUTRAL** (<https://climate.stripe.com/Tmn5wf>)

Disclaimer: Meta is exclusively focused on blockchain education. We do not engage in any cryptocurrency-related activities, nor do we provide advice or guidance in the cryptocurrency field.

Meta is a pioneer in education and career transformation, specializing in today's most in-demand skills. The leading source for training, staffing, and career transitions, we foster a flourishing community of professionals pursuing careers they love.

© 2025 Edmore (Private) Limited

